

Asynchronous API

Below, we showcase different variations of asynchronous APIs built on top of C-style callbacks with examples of doing 2 GET requests - both sequentially and concurrently.

NO threads are involved intentionally, to disconnect any associations of coroutines or fibers with multithreading. Some parts are deliberately simple, while still presenting as much details as possible.

Jump to tasks, std::future, coroutines, fibers, senders. TODO: App_X.

Work In Progress.

Contents

1	# introduction	2
2	# setup with CMake + libcurl	4
3	# building blocking API	5
3.1	# run simple http server for tests	7
4	# building C-style callbacks API	7
4.1	# thoughts on the design	7
4.2	# note on C-style API (vs C++)	9
4.3	# implementing with libcurl multi	9
5	# building std::future API	14
6	# building C++20 coroutines API	16
6.1	# C++ coroutines, basic task type	17
6.2	# C++ coroutines, basic await	22
6.3	# C++ coroutines, await callback with a crash	24
6.4	# C++ coroutines, await callback	27
6.5	# coroutine task that holds a return value	30
6.6	# awaiting coroutine task	33
6.7	# waiting for multiple coroutines	37

7	# building Fibers API	41
7.1	# basic Fiber	42
7.2	# switching between Fibers	45
7.3	# tasks for Fibers	48
7.4	# cancellable Fiber task with a generic callback	49
7.5	# basic FiberPool	53
7.6	# FiberTask	55
8	# senders	62
9	# reactive streams	62

1 # introduction

We start with a simple C-style API on top of libcurl C API and have a code that may look like this:

```

1 // our CURL API
2 std::string CURL_get(const std::string& url);
3
4 int main()
5 {
6     const std::string r1 = CURL_get("localhost:5001/file1.txt");
7     const std::string r2 = CURL_get("localhost:5001/file2.txt");
8     std::println("{} ", r1);
9     std::println("{} ", r2);
10 }
```

The code above performs two GET requests sequentially. Everything executes synchronously.

Next, lets have a simple C-style callbacks API (**not** the C++ one, see the note), to run requests concurrently:

```

1 // libcurl bookkeeping
2 using CURL_Async = void*;
3 CURL_Async CURL_async_create();
4 void CURL_async_destroy(CURL_Async curl_async);
5 void CURL_async_tick(CURL_Async curl_async);
6
7 // main async callback API
8 void CURL_async_get(CURL_Async curl_async
9     , const std::string& url
10    , void* user_data
11    , void (*callback)(void* user_data, std::string response));
```

Doing 2 GET requests is more involved now:

```
1  int main()
2  {
3      struct State
4      {
5          int count = 0;
6          std::string r1;
7          std::string r2;
8      };
9
10     CURL_Async curl_async = CURL_async_create();
11     State state;
12     CURL_async_get(curl_async, "localhost:5001/file1.txt", &state
13         , [](void* user_data, std::string response)
14     {
15         State& state = *static_cast<State*>(user_data);
16         state.count += 1;
17         state.r1 = std::move(response);
18     });
19     CURL_async_get(curl_async, "localhost:5001/file2.txt", &state
20         , [](void* user_data, std::string response)
21     {
22         State& state = *static_cast<State*>(user_data);
23         state.count += 1;
24         state.r2 = std::move(response);
25     });
26     while (state.count != 2) // wait for 2 requests to finish
27     {
28         CURL_async_tick(curl_async);
29     }
30     CURL_async_destroy(curl_async);
31     std::println("{} ", state.r1);
32     std::println("{} ", state.r2);
33 }
```

There is a need to have a `State` for bookkeeping, pass it as a `void*` user data to access later and, finally, run an event loop to give libcurl a chance to process requests. Note, however, requests execute concurrently now, as in - 2 requests are active at the same time.

After this, let's build tasks, `std::future`, coroutines, fibers, senders and other variations of asynchronous API on top of C-style callbacks above.

But before that, let's wrap libcurl C API for our needs.

2 # setup with CMake + libcurl

CODE: CH00_cmake

For vcpkg, there is an extensive documentation available. In short:

```
1 git clone https://github.com/microsoft/vcpkg
2 cd vcpkg
3 bootstrap-vcpkg.bat
```

Assuming vcpkg is installed at K:\vcpkg, we also set VCPKG_ROOT and add this path to PATH environment variable so build scripts can use VCPKG_ROOT and vcpkg without a need to know the exact location.

```
1 set VCPKG_ROOT=K:\vcpkg
2 set PATH=%VCPKG_ROOT%;%PATH%
```

For the project (lets say in a K:\async_api folder), vcpkg manifest mode is used. Together with curl setup, all required steps are

```
1 cd K:\async_api
2 vcpkg new --application
3 vcpkg add port curl
```

Note that to find exact curl package name, vcpkg search curl was used which prints:

```
curl 8.13.0#1 A library for transferring data with URLs
```

CMakeLists.txt now looks like this:

```
1 cmake_minimum_required(VERSION 3.24 FATAL_ERROR)
2 project(async_api LANGUAGES CXX)
3
4 add_executable(00_cmake_libcurl main.cc)
5 target_compile_features(00_cmake_libcurl PUBLIC cxx_std_23)
6 find_package(CURL REQUIRED)
7 target_link_libraries(00_cmake_libcurl PRIVATE CURL::libcurl)
```

find_package(CURL REQUIRED) syntax together with CURL::libcurl target name is found from the output log of vcpkg install curl (or during CMake configuration run) which prints:

```
1 curl is compatible with built-in CMake targets:
2
3     find_package(CURL REQUIRED)
4     target_link_libraries(main PRIVATE CURL::libcurl)
```

To test that everything compiles and links, save main.cc:

```

1  #include <curl/curl.h>
2
3  int main()
4  {
5      CURL* curl = curl_easy_init();
6      assert(curl);
7      curl_easy_cleanup(curl);
8  }

```

Finally, to invoke CMake configure, build and run (with vcpkg):

```

1  cd K:\async_api
2  cmake -S . -B __build ^
3      -DCMAKE_TOOLCHAIN_FILE=%VCPKG_ROOT%\scripts\buildsystems\vcpkg.cmake
4  cmake --build __build --config Debug
5  :: run a test
6  .\__build\CH00_cmake\Debug\CH00_cmake.exe

```

This assumes `cmake.exe` is in your PATH, see `build.cmd`.

3 # building blocking API

CODE: CH01_libcurl_easy

Blocking, synchronous API for a GET request is straightforward. We go with a function that looks like this:

```

1  std::string CURL_get(const std::string& url);

```

libcurl comes with two different APIs, “easy” and “multi”. Lets use an easy interface; libcurl examples available online, including official `simple.c` example for a start.

Everything together leads to the implementation below, where `curl_easy_perform()` call is the main one that blocks the execution until request completes; once complete, we can return results:

```

1  #include <string>
2
3  #include <curl/curl.h>
4
5  #if defined(NDEBUG)
6  #   undef NDEBUG
7  #endif
8  #include <cassert>
9
10 static size_t CURL_OnWriteCallback(
11     void* ptr, size_t size, size_t nmemb, void* data)

```

```

12 {
13     std::string& response = *static_cast<std::string*>(data);
14     response.append(static_cast<const char*>(ptr), size * nmemb);
15     return (size * nmemb);
16 }
17
18 std::string CURL_get(const std::string& url)
19 {
20     CURL* curl = curl_easy_init();
21     assert(curl);
22
23     CURLcode status = curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
24     assert(status == CURLE_OK);
25     status = curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);
26     assert(status == CURLE_OK);
27
28     std::string response;
29     status = curl_easy_setopt(curl
30         , CURLOPT_WRITEFUNCTION, CURL_OnWriteCallback);
31     assert(status == CURLE_OK);
32     status = curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
33     assert(status == CURLE_OK);
34
35     status = curl_easy_perform(curl);
36     assert(status == CURLE_OK);
37
38     long response_code = -1;
39     status = curl_easy_getinfo(curl, CURLINFO_RESPONSE_CODE, &response_code);
40     assert(status == CURLE_OK);
41     assert(response_code == 200L);
42
43     curl_easy_cleanup(curl);
44     return response;
45 }

```

Note on error handling: for now, we crash on any unexpected error - as in “crash the whole application”. `assert()` is enabled always **intentionally** to simplify both, the sample code **and** debugging:

```

1 // after all includes, main.cc
2 #if defined(NDEBUG)
3 #   undef NDEBUG
4 #endif
5 #include <cassert>

```

This is “bad” for generic, low-level library API/code, but could be fine sometimes.

We'll discuss error handling later.

To see the code in action, lets run our program:

```
1 #include <print>
2
3 int main()
4 {
5     const std::string r = CURL_get("localhost:5001/file1.txt");
6     std::println("CURL_get(file1.txt): '{}'", r);
7 }
```

that.. should crash since we don't have local HTTP server running to serve localhost:5001/file1.txt. See the next section on how to make it happen.

Once done, we should see the sample file1.txt content in the console output:

```
CURL_get(file1.txt): 'content 1'
```

3.1 # run simple http server for tests

CODE: CH01_libcurl_easy

To run sample code, lets use Python to have simple HTTP server that hosts files in the current directory, see `serve.cmd`:

```
1 python -m http.server 5001
```

Given the directory that has file1.txt and file2.txt, `CURL_get("localhost:5001/file1.txt")` should work and return the content of the file, see blocking libcurl section.

4 # building C-style callbacks API

CODE: CH02_libcurl_multi

4.1 # thoughts on the design

Now, lets imagine simplest possible asynchronous API. The difference to blocking API is that we ask the system to start a GET request and the response could arrive some time later. The system invokes a user-provided `callback` to notify us once everything is done:

```
1 void CURL_async_get(const std::string& url
2     , void (*callback)(std::string));
```

we could use it like this:

```
1 // start a request:
2 CURL_async_get("localhost:5001/file2.txt"
3     , [](std::string response)
```

```

4 {
5     // probably, some time later:
6     std::println("got response: {}", response);
7 });

```

There are multiple issues with the design above:

1. Where is the “system” that starts the request? It could be implicit, hidden global singleton, but we can also ask a user to explicitly create and pass it around.
2. The callback accepts only **response**, there is no way for a user to access other data, without resorting to global singletons again. When starting a request, user should be able to provide opaque pointer to some data that system does not touch and simply gives it back in callback.
3. When and from where the “system” invokes a **callback**? There are multiple answers, but we go with user-controlled event loop that drives everything.

To solve first issue, let's have explicit API to create and destroy the system:

```

1 using CURL_Async = void*; // system's state
2
3 CURL_Async CURL_async_create();
4 void CURL_async_destroy(CURL_Async curl_async);

```

where **CURL_Async** is the system itself, since user does not care what's that exactly, it's hidden under **void***. User creates the system, uses it and, once not needed, destroys to clean up resources, if any.

To drive a system with event loop, user must call the next API:

```

1 void CURL_async_tick(CURL_Async curl_async);

```

This is the chance for a system to actually do some work over time **and** invoke user-provided callbacks, if needed.

Lastly, to give a user some control over data in the callback, we pass opaque **void*** pointer around:

```

1 // main async callback API
2 void CURL_async_get(CURL_Async curl_async
3     , const std::string& url
4     , void* user_data
5     , void (*callback)(void* user_data, std::string response));

```

user_data could be anything, system gives it back when invoking **callback**. This is user responsibility to ensure that pointer is valid all the time while request is in progress.

There are more nuances, like how frequently/when **CURL_async_tick()** should

be invoked by a user; but all this is left out of the scope.

Overall, everything included, we need to implement the next API, see below:

```
1 // libcurl bookkeeping
2 using CURL_Async = void*;
3 CURL_Async CURL_async_create();
4 void CURL_async_destroy(CURL_Async curl_async);
5 void CURL_async_tick(CURL_Async curl_async);
6
7 // main async callback API
8 void CURL_async_get(CURL_Async curl_async
9     , const std::string& url
10    , void* user_data
11    , void (*callback)(void* user_data, std::string response));
```

4.2 # note on C-style API (vs C++)

For C-style API above, with C++, “the system” could be a class, callback could be `std::function<>` to accept anything, generally making it less verbose, having something like this:

```
1 // the API:
2 class CURL_Async
3 {
4 public:
5     void get(const std::string& url, std::function<void (std::string)>);
6     void tick();
7 };
8
9 // the use:
10 CURL_Async curl;
11 curl.get("localhost:5001/file1.txt", [](std::string r)
12 {
13     std::println("{} ", r);
14 });
15 curl.tick(); // etc
```

However, C-style API we have, is a de-facto standard, familiar and recognized for asynchronous APIs with callbacks (citation needed).

The rest of asynchronous APIs implementations below are built on top of C-style callback API, as a basic building block to cover similar callbacks-based APIs.

4.3 # implementing with libcurl multi

CODE: CH02_libcurl_multi

For our callbacks API:

```
1 using CURL_Async = void*;
2 CURL_Async CURL_async_create();
3 void CURL_async_destroy(CURL_Async curl_async);
4 void CURL_async_tick(CURL_Async curl_async);
5 void CURL_async_get(CURL_Async curl_async
6     , const std::string& url
7     , void* user_data
8     , void (*callback)(void* user_data, std::string response));
```

internally, lets have CURL_AsyncScheduler class to handle adding requests, updating/ticking libcurl event loop and, in general, to represent our whole CURL_Async system state:

```
1 struct CURL_AsyncScheduler
2 {
3     CURL_AsyncScheduler();
4     ~CURL_AsyncScheduler();
5     // no copy, no move
6     CURL_AsyncScheduler(const CURL_AsyncScheduler&) = delete;
7
8     using Callback = std::function<void (CURL* curl_easy)>;
9
10    void tick();
11    void add_request(CURL* curl_easy, Callback on_finish);
12
13    // our state
14    CURLM* _multi_curl = nullptr;
15    std::unordered_map<CURL*, Callback> _curl_to_callback;
16 };
```

This is what we'll return to a user as CURL_Async pointer. Lets do it:

```
1 CURL_Async CURL_async_create()
2 {
3     CURL_AsyncScheduler* scheduler = new(std::nothrow) CURL_AsyncScheduler();
4     assert(scheduler);
5     return scheduler;
6 }
7
8 void CURL_async_destroy(CURL_Async curl_async)
9 {
10    assert(curl_async);
11    CURL_AsyncScheduler* scheduler =
12        static_cast<CURL_AsyncScheduler*>(curl_async);
13    delete scheduler;
14 }
```

Now, user just needs to pass `CURL_Async` handle around. Before implementing internals, lets have a helper function that gets actual `CURL_AsyncScheduler` instance from opaque handle:

```
1 CURL_AsyncScheduler& CURL_scheduler(CURL_Async curl_async)
2 {
3     CURL_AsyncScheduler* scheduler =
4         static_cast<CURL_AsyncScheduler*>(curl_async);
5     assert(scheduler);
6     return *scheduler;
7 }
```

It's not exposed to the user in any way. Lets implement our main API in terms of our internal scheduler:

```
1 void CURL_async_get(CURL_Async curl_async
2     , const std::string& url
3     , void* user_data
4     , void (*callback)(void* user_data, std::string response))
5 {
6     // 1. setup curl easy handle
7     CURL* curl_easy = curl_easy_init();
8     assert(curl_easy);
9     CURLcode status = curl_easy_setopt(curl_easy, CURLOPT_URL, url.c_str());
10    assert(status == CURLE_OK);
11    status = curl_easy_setopt(curl_easy, CURLOPT_FOLLOWLOCATION, 1L);
12    assert(status == CURLE_OK);
13
14    // 2. write response data to separate std::string
15    std::string* state = new std::string{};
16    status = curl_easy_setopt(curl_easy
17        , CURLOPT_WRITEFUNCTION, CURL_OnWriteCallback);
18    assert(status == CURLE_OK);
19    status = curl_easy_setopt(curl_easy, CURLOPT_WRITEDATA, state);
20    assert(status == CURLE_OK);
21
22    // 3. associate with multi handle/event loop
23    CURL_scheduler(curl_async).add_request(curl_easy
24        , [state, user_data, callback](CURL* curl_easy)
25    {
26        long response_code = -1;
27        const CURLcode status = curl_easy_getinfo(curl_easy
28            , CURLINFO_RESPONSE_CODE, &response_code);
29        assert(status == CURLE_OK);
30        assert(response_code == 200L);
31        curl_easy_cleanup(curl_easy);
32        std::string data = std::move(*state);
```

```

33         delete state;
34         callback(user_data, std::move(data));
35     });
36 }

```

There are few moving parts and issues:

1. we create and setup curl easy handle in the same way as for blocking call;
2. we allocate separate `std::string` to write the response data to with the same `CURL_OnWriteCallback` callback as in the blocking implementation;
3. finally, we associate the request with event loop/multi handle
4. new `std::string` will leak the memory if the request is not completed
5. overall, there are more hidden allocations from within `add_request()`:
 - a) `std::function<>` most likely allocates
 - b) `std::unordered_map` allocates

It could be done another way around, eliminating the need for separate `std::string` allocation and few more optimizations, mainly with the help of associating user data with curl easy handle/ `CURLOPT_PRIVATE`. However, it's good enough for illustrative purposes.

After creation of curl easy handle, we associate it with curl multi handle:

```

1  CURL_AsyncScheduler::CURL_AsyncScheduler()
2  {
3      const CURLcode status = curl_global_init(CURL_GLOBAL_ALL);
4      assert(status == CURLE_OK);
5      _multi_curl = curl_multi_init();
6      assert(_multi_curl);
7  }
8
9  CURL_AsyncScheduler::~CURL_AsyncScheduler()
10 {
11     const CURLMcode status = curl_multi_cleanup(_multi_curl);
12     assert(status == CURLM_OK);
13     curl_global_cleanup();
14 }
15
16 void CURL_AsyncScheduler::add_request(CURL* curl_easy, Callback on_finish)
17 {
18     assert(on_finish);
19     assert(curl_easy);
20     assert(!_curl_to_callback.contains(curl_easy));
21
22     const CURLMcode status = curl_multi_add_handle(_multi_curl, curl_easy);
23     assert(status == CURLM_OK);
24     _curl_to_callback[curl_easy] = std::move(on_finish);

```

25 }

`_curl_to_callback` map is used to be able to retrieve callback later, given curl easy handle (CURL*).

Our user-exposed `CURL_async_tick()` API is implemented in terms of scheduler:

```
1 void CURL_async_tick(CURL_Async curl_async)
2 {
3     CURL_scheduler(curl_async).tick();
4 }
5
6 void CURL_AsyncScheduler::tick()
7 {
8     int running_handles = -1;
9     CURLMcode status = curl_multi_perform(_multi_curl, &running_handles);
10    assert(status == CURLM_OK);
11    int msgs_in_queue = 0;
12    while (CURLMsg* m = curl_multi_info_read(_multi_curl, &msgs_in_queue))
13    {
14        if (m->msg != CURLMSG_DONE)
15        {
16            continue;
17        }
18        CURL* curl_easy = m->easy_handle;
19        assert(curl_easy);
20        status = curl_multi_remove_handle(_multi_curl, curl_easy);
21        assert(status == CURLM_OK);
22        auto it = _curl_to_callback.find(curl_easy);
23        assert(it != _curl_to_callback.end());
24        Callback callback = std::move(it->second);
25        assert(callback);
26        (void)_curl_to_callback.erase(it);
27        callback(curl_easy);
28    }
29 }
```

The main part of event loop is the call to `curl_multi_perform()`. Once done we ask for easy handle requests that were completed, search for an associated callback for each request and invoke it.

Note, there are no threads involved and it's possible to create many GET requests at once with multiple calls to `CURL_async_get()` - libcurl will manage them all together.

Again, it's user responsibility to drive libcurl with a periodic calls to `CURL_async_tick()`. Lets do single request with the API above:

```

1  #include <print>
2
3  int main()
4  {
5      struct State
6      {
7          std::string response;
8          bool done = false;
9      };
10     CURL_Async curl_async = CURL_async_create();
11     State state;
12     CURL_async_get(curl_async, "localhost:5001/file1.txt", &state
13         , [](void* user_data, std::string response)
14     {
15         State& state = *static_cast<State*>(user_data);
16         state.response = std::move(response);
17         state.done = true;
18     });
19     while (!state.done)
20     {
21         CURL_async_tick(curl_async);
22     }
23     CURL_async_destroy(curl_async);
24
25     std::println("async response: '{}'", state.response);
26 }

```

If python HTTP server is running, our program should print:

```

1  async response: 'content 1'

```

5 # building std::future API

CODE: CH0x_future

Having CURL_async_get() leads to the next implementation that wraps everything into a std::future:

```

1  std::future<std::string> future_async_get(
2      CURL_Async curl_async, const std::string& url)
3  {
4      using Promise = std::promise<std::string>;
5      Promise* promise = new(std::nothrow) Promise;
6      assert(promise);
7      std::future<std::string> future = promise->get_future();
8      CURL_async_get(curl_async, url, promise

```

```

9      , [](void* user_data, std::string response)
10     {
11         Promise* promise = static_cast<Promise*>(user_data);
12         promise->set_value(std::move(response));
13         delete promise;
14     });
15     return future;
16 }

```

Everything is just what `std::` exposes. Note how we need to allocate our promise so it can be alive until the end of the request.

All in all, we:

1. allocate `std::promise`
2. `std::promise` allocates shared state, used by `std::future`
3. `CURL_async_get` allocates `std::string` to write a response
4. `CURL_async_get` allocates `std::function` for a generic callback
5. `CURL_async_get` allocates `std::unordered_map` node to remember what to call when
6. .. and probably something else (CURL internals, etc)

Anyway, `std::future` is missing useful bits to work with it in a non-blocking manner:

- no built-in `.is_ready()` check
- no built-in `.then()` continuation, see Design and evolution of C++ future continuations

Faking `is_future_ready()` with:

```

1 template<typename T>
2 bool is_future_ready(const std::future<T>& future)
3 {
4     const std::future_status status = future.wait_for(std::chrono::seconds::zero());
5     return (status == std::future_status::ready);
6 }

```

allows to finally write something among the lines:

```

1 int main()
2 {
3     CURL_Async curl_async = CURL_async_create();
4     std::future<std::string> result = future_async_get(
5         curl_async, "localhost:5001/file1.txt");
6     while (is_future_ready(result) == false)
7     {
8         CURL_async_tick(curl_async);
9     }

```

```

10     CURL_async_destroy(curl_async);
11     std::println("async future response: '{}'", result.get());
12 }

```

Missing `std::future` features makes it not composable; waiting 2 tasks to finish is the same as checking 2 flags to become true; giving not much of a win compared to direct use of `CURL_async_get()`.

See `App_Futures`, `App_Callbacks`.

blocking, synchronous (`App_Blocking`)

on error handling

assume success always (tooling)

implicit, return empty string

status code, out parameter (`std::filesystem-style`)

optional

exceptions

result/variant-like

result/tuple-like

result/specialized

async polling, tasks (`App_Tasks`)

blocking `std::future`/promise

async polling, `std::future`/promise

async, callbacks (`App_Callbacks`)

async, callbacks + polling (tasks, handle)

async with statefull/implicit callback (`state.on_X.subscribe/delegate`)

6 # building C++20 coroutines API

Coroutines materials:

- How C++ coroutines work.
- All of Asymmetric Transfer, author of `cppcoro`.

In short, we'd like to be able to write something like this:

```
1  const std::string response = co_await CURL_await_get(  
2      curl_async, "localhost:5001/file1.txt");  
3  // use `response` as a usual variable, no callbacks
```

There are several moving and a bit unrelative parts to have working coroutines code. First, coroutine function return type needs to be built, just to be able to write any/empty coroutine:

```
1  Co_Task coro_work()  
2  {  
3      co_return;  
4  }
```

Next, there is a need to write coroutine awaitable to be able to `co_await` some work, specifically, GET request:

```
1  Co_Task coro_work(CURL_Async curl_async)  
2  {  
3      std::string response = co_await CURL_await_get(curl_async  
4          , "localhost:5001/file1.txt");  
5      co_return;  
6  }
```

And, finally, there are some challenges to have a code that has several GET requests on the fly, with coroutines.

Lets start with basics.

6.1 # C++ coroutines, basic task type

CODE: CH0x_coro_basic_task

There is a trick to writing some basic C++20 coroutines code - **listen to the compiler**. Lets see what it takes to make the next code “work”:

```
1  Co_Task coro_work()  
2  {  
3      co_return;  
4  }
```

`Co_Task` is a class, lets have empty one and try to compile:

```
1  struct Co_Task {};  
2  
3  Co_Task coro_work()  
4  {  
5      co_return;  
6  }
```

MSVC complains:

```
1 main.cc(164,5): error C3774: cannot find 'std::coroutine_traits':  
2           Please include <coroutine> header
```

after including <coroutine> header:

```
1 main.cc(166,5): error C2039: 'promise_type': is not a member of  
2           'std::coroutine_traits<Co_Task>'
```

Lets add empty `promise_type` class inside `Co_Task`:

```
1 #include <coroutine>  
2  
3 struct Co_Task  
4 {  
5     struct promise_type {};  
6 };  
7  
8 Co_Task coro_work()  
9 {  
10     co_return;  
11 }
```

MSVC complains:

```
1 main.cc(170,1): error C3789: this function cannot be a coroutine:  
2           'Co_Task::promise_type' does not declare the member  
3           'get_return_object()'  
4 main.cc(170,1): error C3789: this function cannot be a coroutine:  
5           'Co_Task::promise_type' does not declare the member  
6           'initial_suspend()'  
7 main.cc(170,1): error C3789: this function cannot be a coroutine:  
8           'Co_Task::promise_type' does not declare the member  
9           'final_suspend()'
```

Ah, so `promise_type` should have `get_return_object()`, `initial_suspend()` and `final_suspend()` member functions. Return types are unclear, unfortunately. To speed-up things, we know that `get_return_object()` should return `Co_Task`. For `initial_suspend()` and `final_suspend()` we'll go with `std::suspend_always` awaitables for now. That gives:

```
1 #include <coroutine>  
2  
3 struct Co_Task  
4 {  
5     struct promise_type  
6     {  
7         Co_Task get_return_object()           { return {}; }  
8     }  
9 }
```

```

8         std::suspend_always initial_suspend() { return {}; }
9         std::suspend_always final_suspend()   { return {}; }
10    };
11 };
12
13 Co_Task coro_work()
14 {
15     co_return;
16 }

```

MSVC complains:

```

1 main.cc(164,12): error C3781: Co_Task::promise_type: a coroutine's
2                   promise must declare either
3                   'return_value' or 'return_void'
4 main.cc(176,1):  error C2039: 'unhandled_exception': is not a member
5                   of 'Co_Task::promise_type'

```

Since our `coro_work()` coroutine has just `co_return`, we should provide `return_void()` member function. With `unhandled_exception()`, we have:

```

1 struct promise_type
2 {
3     Co_Task get_return_object()          { return {}; }
4     std::suspend_always initial_suspend() { return {}; }
5     std::suspend_always final_suspend()  { return {}; }
6     void return_void()                  {}
7     void unhandled_exception()           {}
8 };

```

MSVC complains:

```

1 main.cc(168,29): error C5231: the expression
2                   'co_await promise.final_suspend()' must be non-throwing

```

OK, makes sense. Finally,

```

1 #include <coroutine>
2
3 struct Co_Task
4 {
5     struct promise_type
6     {
7         Co_Task get_return_object()          { return {}; }
8         std::suspend_always initial_suspend() { return {}; }
9         std::suspend_always final_suspend() noexcept { return {}; }
10        void return_void()                  {}
11        void unhandled_exception()           {}

```

```

12     };
13 };
14
15 Co_Task coro_work()
16 {
17     co_return;
18 }

```

compiles! We just need to fill in details and implement given functions properly.

There are way too many different ways to implement coroutine task/promise types. There are no constraints and, in general, it all depends on your design and needs. We'll go with an owning coroutine task type:

1. `Co_Task` will own coroutine handle (as in free coroutine in the destructor).
2. Because of the above, `final_suspend()` must suspend always.
3. `Co_Task` will be a “lazy” coroutine, meaning, it's going to be suspended after initial call of `coro_work()/coroutine` function.
4. Because of the above, `initial_suspend()` must suspend.
5. Because coroutine is suspended initially, `Co_Task` needs to expose `resume()` or similar function to run a coroutine.

For now, lets proceed with implementation. Since we own coroutine, our `Co_Task` needs to have destructor, should be move-only:

```

1 struct Co_Task
2 {
3     struct promise_type;
4     using co_handle = std::coroutine_handle<promise_type>;
5
6     struct promise_type
7     {
8         Co_Task get_return_object()
9         {
10             return Co_Task{co_handle::from_promise(*this)};
11         }
12         // ...
13     };
14
15     Co_Task(co_handle coro)
16         : _coro{coro} {}
17     Co_Task(Co_Task&& rhs) noexcept
18         : _coro{std::exchange(rhs._coro, {})} { }
19     Co_Task(const Co_Task&) = delete;
20     ~Co_Task() noexcept
21     {
22         if (_coro)

```

```

23         {
24             _coro.destroy();
25         }
26     }
27
28     co_handle _coro;
29 };

```

In short, when we call `coro_work()`, compiler creates `Co_Task::promise_type` and invokes `get_return_object()` to be able to return an instance of `Co_Task` to the user. Here, in `get_return_object()` there is a way to get an access to `std::coroutine_handle<>` - the only way to interact with just allocated coroutine. Once `Co_Task` is created, we return it to the user. It's **up to the user** to manage `std::coroutine_handle<>`. In our case, we own just created coroutine, hence if `Co_Task` is destroyed, we assume coroutine is in suspended state and destroy it too.

Writing down the rest of functions:

```

1  std::suspend_always promise_type::initial_suspend()
2  {
3      return {};
4  }
5
6  std::suspend_always promise_type::final_suspend() noexcept
7  {
8      return {};
9  }
10
11 void promise_type::return_void()
12 {
13     // yeah, we return void. Nothing to do
14 }
15
16 void promise_type::unhandled_exception()
17 {
18     // crash, no exceptions handling
19     assert(false);
20 }

```

we can test the basics:

```

1  Co_Task coro_work()
2  {
3      std::println("inside coro_work");
4      co_return;
5  }

```

```

6
7  int main()
8  {
9      Co_Task coro = coro_work();
10 }

```

which runs and... prints nothing since our coroutine is created and immediately suspended even before executing first print.

Lets expose `resume()` for our `Co_Task` and use it:

```

1 void Co_Task::resume()
2 {
3     assert(!_coro);
4     assert(!_coro.done());
5     _coro.resume();
6 }
7
8 Co_Task coro_work()
9 {
10     std::println("inside coro_work");
11     co_return;
12 }
13
14 int main()
15 {
16     std::println("-- before coro_work()");
17     Co_Task coro = coro_work();
18     std::println("-- after coro_work()");
19     coro.resume();
20     std::println("-- after resume()");
21 }

```

which prints:

```

-- before coro_work()
-- after coro_work()
inside coro_work
-- after resume()

```

6.2 # C++ coroutines, basic await

CODE: CH0x_coro_basic_await

Given that we can have simplest coroutine, what does it take to `co_await`? Lets try to compile:

```

1 struct Co_CurlAsync {};
2

```

```

3  Co_Task coro_work()
4  {
5      co_await Co_CurlAsync{};
6      co_return;
7  }

```

MSVC complains:

```

1  main.cc(73,26): error C2039: 'await_ready': is not a member of 'Co_CurlAsync'
2  main.cc(73,26): error C2039: 'await_suspend': is not a member of 'Co_CurlAsync'
3  main.cc(70,26): error C2039: 'await_resume': is not a member of 'Co_CurlAsync'

```

So `co_await` requires “awaiter” to have those 3 functions. We can think about awriter as something that:

1. knows if some operation is ready or not (`..._ready`)
2. knows how to resume coroutine later (`..._suspend`)
3. knows how to get the result of awaited operation (`..._resume`)

The compiler asks awriter, specifically, `Co_CurlAsync` with `bool await_ready()` if operation is done/ready or is in progress. If awriter returns false, the compiler switches current coroutine state to “suspended” and invokes awriter’s `await_suspend(std::coroutine_handle<> coro)` customization point which allows to remember currently suspended coroutine `coro` handle, to call `.resume()` later, once operation is done. Once coroutine is resumed, compiler asks for a value from last awriter responsible for suspend.

In short, we can have `Co_CurlAsync` awriter that tells that (1) operation is not ready yet (2) on suspend, resumes coroutine immediately and (3) returns nothing:

```

1  struct Co_CurlAsync
2  {
3      bool await_ready()
4      {
5          return false;
6      }
7
8      void await_suspend(std::coroutine_handle<> coro)
9      {
10         std::println("-- inside suspend, resuming immediately");
11         coro.resume();
12     }
13
14     void await_resume()
15     {
16         std::println("-- resume");
17     }

```

```

18 };
19
20 Co_Task coro_work()
21 {
22     std::println("before co_await");
23     co_await Co_CurlAsync{};
24     std::println("after co_await");
25     co_return;
26 }
27
28 int main()
29 {
30     Co_Task coro = coro_work();
31     coro.resume();
32 }

```

which prints:

```

before co_await
-- inside suspend, resuming immediately
-- resume
after co_await

```

Now, on suspend, we did nothing, but immediately resumed coroutine. But we also could start an async operation and, on finish, resume the coroutine.

6.3 # C++ coroutines, await callback with a crash

CODE: CH0x_coro_curl0

Lets continue implementing Co_CurlAsync above, in short:

```

1 struct Co_CurlAsync
2 {
3     CURL_Async _curl_async{};
4     std::string _url;
5     std::coroutine_handle<> _coro;
6     std::string _response;
7
8     bool await_ready()
9     { // 1. CURL_async_get() is not yet started, force coroutine suspend:
10         return false;
11     }
12
13     void await_suspend(std::coroutine_handle<> coro)
14     { // 2. remember coroutine handle, start request, resume on finish:
15         _coro = coro;
16     }

```



```

17         CURL_async_get(_curl_async, _url, this
18             , [](void* user_data, std::string response)
19         {
20             Co_CurlAsync& self = *static_cast<Co_CurlAsync*>(user_data);
21             self._response = std::move(response);
22             self._coro.resume();
23         });
24     }
25
26     std::string await_resume()
27     { // 3. after resume, return response:
28         return std::move(_response);
29     }
30 };
31
32 Co_CurlAsync CURL_await_get(CURL_Async curl_async, const std::string& url)
33 {
34     return Co_CurlAsync{._curl_async = curl_async, ._url = url};
35 }

```

So, now `co_await CURL_await_get(..., "url")` should compile and kind-a work. As always, there are few moving part.

When coroutine function (represented as `std::coroutine_handle<>`) `co_await`s our CURL awaiter - `Co_CurlAsync`, we:

1. force whole coroutine to suspend, since we return false from `await_ready()`
2. this is needed so compiler invokes `await_suspend()` and gives us a handle to currently awaiting coroutine, so we can (a) start request and (b) resume coroutine with a call to `coro.resume()`
3. finally, once request is complete, we can return the `_response` from `await_resume()`

There is one big issue there: what if we start a request with `CURL_async_get()`, coroutine suspends, BUT user discards `Co_Task` value that destroys coroutine, making `std::coroutine_handle<>` we remembered - dangling? There are several possible solutions, but lets see the current code in action by writing our `main()` function:

```

1 Co_Task coro_main(CURL_Async curl_async)
2 {
3     const std::string response = co_await CURL_await_get(
4         curl_async, "localhost:5001/file1.txt");
5
6     std::println("coro_main response: '{}'", response);
7     co_return;
8 }

```

```

9
10 int main()
11 {
12     CURL_Async curl_async = CURL_async_create();
13     Co_Task task = coro_main(curl_async);
14     task.resume();
15     while (task.is_in_progress())
16     {
17         CURL_async_tick(curl_async);
18     }
19     CURL_async_destroy(curl_async);
20 }

```

Here, we setup CURL_Async, as usual, and drive the loop until coroutine is in progress:

```

1 bool Co_Task::is_in_progress() const
2 {
3     assert(!_coro);
4     return !_coro.done();
5 }

```

Running the sample should print:

coro_main response: 'content 1'

However, that works because we wait for coroutine until full complete. If we discard Co_Task too early, there is going to be a crash:

```

1 int main()
2 {
3     CURL_Async curl_async = CURL_async_create();
4
5     {
6         Co_Task task = coro_main(curl_async);
7         task.resume(); // run
8     } // **destroy**
9
10    while (true)
11    {
12        CURL_async_tick(curl_async); // resume coroutine from there
13    }
14    CURL_async_destroy(curl_async);
15 }

```

It happens because CURL_async_get() callback remembers 2 pointers:

1. this pointer to Co_CurlAsync/awaiter which is owned by coroutine frame

2. and `coroutine_handle<>` itself, which we destroy BEFORE `CURL_async_get()` finish.

In short, we start request, then `.destroy()` coroutine, then try to resume dangling coroutine inside a callback with a call to `.resume()` even using stale pointer to awaiter (user data in the callback).

There are several solutions, few of them:

1. Don't own and don't destroy coroutine inside `Co_Task` destructor (.. in a multiple ways).
2. Delay coroutine destroy if there are live references to it.
3. Be able to cancel `CURL_async_get()` request if coroutine/awaiter is destroyed.
4. Ensure that callback has a safe way to detect dead coroutine and do nothing.

1st solution could be the best but changes completely the semantics of `Co_Task`, does not allow to easily have `Co_Task<T>` that return some value and requires to be able to change `Co_Task` internals.

2nd solution is similar in the sense that it also requires `Co_Task` changes and the code around.

3rd solution requires changes to our basic C-style callback API which we assume we can't do (since, otherwise, the interface is more advanced).

4th solution is the most inefficient and requires no changes neither in `Co_Task` nor in callback API.

6.4 # C++ coroutines, await callback

CODE: CH0x_coro_curl

Lets fix the problematic part in a simple way:

```

1 // struct Co_CurlAsync ...
2 void await_suspend(std::coroutine_handle<> coro)
3 { // 2. remember coroutine handle, start request, resume on finish:
4     _coro = coro;
5
6     CURL_async_get(_curl_async, _url
7         , this // ** HERE
8         , [](void* user_data, std::string response)
9         {
10             Co_CurlAsync& self = *static_cast<Co_CurlAsync*>(user_data);
11             self._response = std::move(response);
12             self._coro.resume();
13         });
14 }
```

For a “happy” path, when `CURL_async_get()` completes before coroutine destruction, the flow is:

- 1) create `Co_CurlAsync`, invoke `CURL_async_get`
- 2) invoke callback (access this/coroutine)
- 3) destroy `Co_CurlAsync`

For a “bad” path, when coroutine is destroyed while we have `CURL_async_get` in progress, the flow is:

- 1) create `Co_CurlAsync`, invoke `CURL_async_get`
- 2) destroy `Co_CurlAsync`
- 3) invoke callback (access this/dead coroutine)

Lets allocate a separate object that can outlive the coroutine/await. There is an assumption that `CURL_async_get()` callback is always going to be invoked. Given this, we:

- A) allocate `WaitState` object - right before `CURL_async_get()`
- B) pass it to the callback instead of `this`
- C) destroy allocated object inside callback

This way we guarantee that the object is always alive while request is in progress and its lifetime is bound the the request itself and nothing else:

```
1  _wait_state = new(std::nothrow) WaitState{._self = this};
2  assert(_wait_state);
3
4  CURL_async_get(_curl_async, _url
5      , _wait_state
6      , [](void* user_data, std::string response)
7  {
8      WaitState* wait_state = static_cast<WaitState*>(user_data);
9      assert(wait_state);
10     if (wait_state->_self)
11     {
12         Co_CurlAsync& self = *wait_state->_self;
13         self._wait_state = nullptr;
14         self._response = std::move(response);
15         self._coro.resume();
16     }
17     // else: Co_CurlAsync/coroutine is dead
18     delete wait_state;
19 });
```

`WaitState` is just a struct that has a reference to `Co_CurlAsync`:

```
1  struct Co_CurlAsync
2  {
3      struct WaitState
```

```

4     {
5         Co_CurlAsync* _self = nullptr;
6     };
7     WaitState* _wait_state = nullptr;

```

When coroutine/Co_CurlAsync is destroyed, we need to mark WaitState reference to it as null so CURL_async_get() knows it's not alive:

```

1 ~Co_CurlAsync()
2 {
3     if (_wait_state)
4     { // CURL_async_get() is still in progress
5         assert(_wait_state->_self == this);
6         _wait_state->_self = nullptr; // dead
7     }
8     // else: CURL_async_get() is already completed
9 }

```

That's how you write inefficient coroutine types for a systems that know nothing about coroutines. When doing simple call:

```

1 const std::string response = co_await CURL_await_get(
2     curl_async, "localhost:5001/file1.txt");

```

we:

1. allocate coroutine frame itself
2. CURL_await_get allocates WaitState
3. CURL_async_get allocates std::string to write a response
4. CURL_async_get allocates std::function for a generic callback
5. CURL_async_get allocates std::unordered_map node to remember what to call when
6. .. and probably something else (CURL internals, etc)

“simple” CURL_async_get() implementation alone brings 3 allocations. “simple” co_await CURL_async_get() brings 2 more separate allocations.

With CURL scheduler that **knows** about coroutines and few more optimizations and limitations, this number of allocations can go down to amortized 0:

- CURL scheduler preallocates up to N max requests
- request itself knows how to store the response and coroutine-callback inline
- coroutine itself re-uses memory pool for up to N max active coroutines.

Anyway, we now can co-await requests with a nice syntax:

```

1 static Co_Task coro_main(CURL_Async curl_async)
2 {
3     const std::string r1 = co_await CURL_await_get(
4         curl_async, "localhost:5001/file1.txt");

```

```

5     const std::string r2 = co_await CURL_await_get(
6         curl_async, "localhost:5001/file2.txt");
7     std::println("{} ", r1);
8     std::println("{} ", r2);
9     co_return;
10 }
11
12 int main()
13 {
14     CURL_Async curl_async = CURL_async_create();
15     Co_Task task = coro_main(curl_async);
16     task.resume();
17     while (task.is_in_progress())
18     {
19         CURL_async_tick(curl_async);
20     }
21     CURL_async_destroy(curl_async);
22 }

```

Note, however, we run 2 requests one after another: second request starts when first `co_await` ends. There is no concurrency..

6.5 # coroutine task that holds a return value

CODE: CH0x_coro_task

To be more useful, we'd like to be able return something out of coroutine:

```

1 static Co_Task<int> coro_main()
2 {
3     co_return 3;
4 }
5
6 int main()
7 {
8     Co_Task<int> task = coro_main();
9     task.resume();
10    std::println("coro main: {}", task.get_once());
11 }

```

It requires implementing `promise_type` `return_value()` and `return_void()` customization points. The only complication is that we need to handle `Co_Task<void>`. To do so, lets start with `promise_return<T>`:

```

1 template<typename T>
2 struct promise_return
3 {

```

```

4     std::variant<std::monostate, T> _value;
5     template<typename U>
6     void return_value(U&& v) noexcept
7     {
8         _value.template emplace<1>(std::forward<U>(v));
9     }
10    const T& get() const noexcept
11    {
12        assert(_value.index() == 1);
13        return std::get<1>(_value);
14    }
15    T&& get_once() noexcept
16    {
17        const T& v = static_cast<const promise_return*>(*this).get();
18        return std::move(const_cast<T*>(v));
19    }
20 };
21
22 template<>
23 struct promise_return<void>
24 {
25     void return_void() noexcept
26     {
27     }
28     void get() const noexcept
29     {
30     }
31     void get_once() noexcept
32     {
33     }
34 };
35
36 template<typename T>
37 struct promise_return<T&>
38 {
39     static_assert(sizeof(T) == 0, "we do not support Co_Task<T&>");
40 };

```

promise_return for void injects return_void(), get() returns nothing. promise_return for any type T injects return_value() and remembers the value. std::variant is used to handle non-default-constructible types and, in general, construct a value only at the point of the actual return/co_return.

With the help of promise_return, promise_type remains the same as our initial version:

```

1  template<typename T>
2  struct Co_Task
3  {
4      struct promise_type;
5      using co_handle = std::coroutine_handle<promise_type>;
6
7      struct promise_type : promise_return<T>
8      {
9          Co_Task get_return_object() noexcept
10         {
11             return Co_Task{co_handle::from_promise(*this)};
12         }
13         std::suspend_always initial_suspend() noexcept
14         {
15             return {};
16         }
17         std::suspend_always final_suspend() noexcept
18         {
19             return {};
20         }
21         void unhandled_exception() noexcept
22         {
23             // crash, no exceptions handling
24             assert(false);
25         }
26     };
27
28     // ...
29     decltype(auto) get() const
30     {
31         assert(_coro);
32         return _coro.promise().get();
33     }
34     decltype(auto) get_once() const
35     {
36         assert(_coro);
37         return _coro.promise().get_once();
38     }
39
40     co_handle _coro;
41 };

```

Now Co_Task<T> works:

```

1  static Co_Task<int> coro_main()
2  {

```



```

3     co_return 3;
4 }

```

6.6 # awaiting coroutine task

CODE: CH0x_coro_await_task

Now, given `Co_Task<T>`, we need to await for it somehow:

```

1 static Co_Task<int> coro_get()
2 {
3     co_return 3;
4 }
5
6 static Co_Task<int> coro_main()
7 {
8     const int v = co_await coro_get();
9     co_return v;
10 }

```

There are 2 important implementation bits to know:

- Symmetric Transfer
- `final_suspend()` can return **custom** awaitable

`final_suspend()` allows to inject custom code for the moment of coroutine END, so we can do something at that point in time:

```

1 struct promise_type : promise_return<T>
2 {
3     auto final_suspend() noexcept
4     {
5         struct Final_Await : std::suspend_always
6         {
7             void await_suspend(co_handle self_coro) noexcept
8             {
9                 // **HERE**
10            }
11        };
12        return Final_Await{};
13    }

```

Given a simple coroutine:

```

1 static Co_Task<int> coro_get()
2 {
3     co_return 3;
4 } // <-- final_suspend()

```

our `Final_Await`, `await_suspend()` gets invoked after `co_return`, passing a coroutine that is about to end. This is exactly what we need to notify some other waiting code that we are done. We could invoke a callback... BUT with a symmetric transfer, `await_suspend()` can return a **next** coroutine that must be executed:

```

1 struct promise_type : promise_return<T>
2 {
3     std::coroutine_handle<> _waiting_coro = std::noop_coroutine();
4
5     auto final_suspend() noexcept
6     {
7         struct Final_Await : std::suspend_always
8         {
9             std::coroutine_handle<> await_suspend(co_handle self_coro) noexcept
10             {
11                 return self_coro.promise()._waiting_coro;
12             }
13         };
14         return Final_Await{};
15     }

```

Here, if no one waits for us, we return `std::noop_coroutine()` that does nothing. Otherwise, if `_waiting_coro` was set, we end the execution of our coroutine and resume any other coroutine that was waiting for us. To setup `_waiting_coro`, we implement awaitable interface for `Co_Task` itself:

```

1 template<typename T>
2 struct Co_Task
3 {
4     bool await_ready()
5     {
6         assert(is_in_progress());
7         return false;
8     }
9     // intentionally auto, not decltype(auto)
10    auto await_resume()
11    {
12        return get_once();
13    }
14    std::coroutine_handle<> await_suspend(std::coroutine_handle<> waiting_coro)
15    {
16        _coro.promise()._waiting_coro = waiting_coro;
17        return _coro; // resume us
18    }
19
20    co_handle _coro;

```

```
21     };
```

It's important to understand what is `_coro` and what is `waiting_coro`. We have 3 moving parts (1) `Final_Await::await_suspend()` with `self_coro`, (2) `Co_Task::await_suspend()` `waiting_coro` and (3) `Co_Task _coro` member itself:

```
1 struct Co_Task
2 {
3     [1] ... promise_type::Final_Await::await_suspend(co_handle self_coro) noexcept
4     {
5         return self_coro.promise()._waiting_coro;
6     }
7     [2] ... await_suspend(std::coroutine_handle<> waiting_coro)
8     {
9         _coro.promise()._waiting_coro = waiting_coro;
10        return _coro; // resume us
11    }
12    [3] co_handle _coro;
13 };
```

Given our awaiting code sample:

```
1 static Co_Task<int> coro_get()
2 {
3     co_return 3;
4 }
5
6 static Co_Task<int> coro_main()
7 {
8     co_return co_await coro_get();
9 }
```

We have 2 coroutines created, where:

1. `_coro` from part [3] is `coro_get()`
2. `waiting_coro` from part [2] `await_suspend()` is our `coro_main()`; and
3. `self_coro` from part [1] is `coro_get()` again that gets finished.

So, we

1. create `coro_main()`
2. create `coro_get()`
3. then `await_suspend()` on `coro_get()` from within `coro_main()`
4. that remembers that `_waiting_coro` is `coro_main()`
5. resumes `coro_get()` execution (by returning it, symmetric transfer); and then
6. we finish `coro_get()` and its `final_suspend()` resumes `_waiting_coro` which is `coro_main()`.

That way `coro_get()` was scheduled to be executed because we `co_await` it from within `coro_main()` and `coro_main()` was executed 2nd time because `coro_get()` was completed.

One note on the `co_await` semantic and ownership: see how for `await_resume()` we use `get_once()` to move the value out awaited task. In general, when `co_awaiting`, we consume the task and it can't be used after `co_await`. So the next code should be disallowed:

```
1 static Co_Task<int> coro_main()
2 {
3     Co_Task<int> t = coro_get(1);
4     co_await t; // t is lvalue
5     t.get(); // error
6 }
```

To enforce that, we use `operator co_await() &&` with `&&` ref-qualifier; see also C++ Coroutines: Understanding operator `co_await`:

```
1 struct Co_Await
2 {
3     Co_Task<T> _task;
4
5     bool await_ready()
6     {
7         assert(_task.is_in_progress());
8         return false;
9     }
10    // intentionally auto, not decltype(auto)
11    auto await_resume()
12    {
13        return _task.get_once();
14    }
15    std::coroutine_handle<> await_suspend(std::coroutine_handle<> waiting_coro)
16    {
17        _task._coro.promise()._waiting_coro = waiting_coro;
18        return _task._coro; // resume us
19    }
20 };
21
22 Co_Await operator co_await() &&
23 {
24     // consume this.
25     return Co_Await{._task{std::move(*this)}};
26 }
```

Now we can `co_await` our tasks multiple times:

```

1 static Co_Task<int> coro_get(int v)
2 {
3     co_return v;
4 }
5
6 static Co_Task<int> coro_main()
7 {
8     const int v1 = co_await coro_get(2);
9     const int v2 = co_await coro_get(3);
10    co_return (v1 + v2);
11 }
12
13 int main()
14 {
15     Co_Task<int> task = coro_main();
16     task.resume();
17     std::println("coro main: {}", task.get_once());
18 }

```

Note, how we still execute `coro_get(2)` first then `coro_get(3)` second. There is still no way to launch those 2 coroutines concurrently and wait for both of them:

```

1 static Co_Task<int> coro_main()
2 {
3     auto [v1, v2] = co_await CO_await_all(coro_get(2), coro_get(3));
4     co_return (v1 + v2);
5 }

```

6.7 # waiting for multiple coroutines

Lets implement awaiting for multiple `Co_Tasks`:

```

1 static Co_Task<int> coro_main()
2 {
3     auto [v1, v2] = co_await CO_await_all(coro_get(2), coro_get(3));
4     co_return (v1 + v2);
5 }

```

Note how we return a tuple of all awaited results (since our coroutine task can never fail). Also, note how this is different to sequential `co_await` since all children coroutine tasks are started at the point of `await` and executed concurrently:

```

1 static Co_Task<void> coro_main()
2 {
3     int v1 = co_await coro_get(2);

```

```

4     int v2 = co_await coro_get(3); // runs strictly AFTER first task
5 }

```

To await N tasks, our parent coroutine/task needs to be resumed only when last of that N tasks completes. To achieve this, we introduce a counter that signals how many tasks are still in progress. So our `Final_Await` can resume the parent only when the wait count is zero. See the old implementation:

```

1 struct promise_type : promise_return<T>
2 {
3     std::coroutine_handle<> _waiting_coro = std::noop_coroutine();
4
5     auto final_suspend() noexcept
6     {
7         struct Final_Await : std::suspend_always
8         {
9             std::coroutine_handle<> await_suspend(co_handle self_coro) noexcept
10             {
11                 return self_coro.promise()._waiting_coro;
12             }
13         };
14         return Final_Await{};
15     }

```

and compare to a new one:

```

1 struct promise_type : promise_return<T>
2 {
3     std::int32_t* _wait_count = nullptr;
4     co_handle _waiting_coro;
5
6     std::coroutine_handle<> Final_Await::await_suspend(co_handle self_coro) noexcept
7     {
8         promise_type& self = self_coro.promise();
9         if (self._waiting_coro)
10         {
11             assert(self._wait_count);
12             std::int32_t& wait_count = *self._wait_count;
13             wait_count -= 1;
14             assert(wait_count >= 0);
15             if (wait_count == 0)
16             {
17                 return self._waiting_coro;
18             }
19             // else: we are not the last task, do nothing.
20         }
21         // else: no one was awaiting us.

```

```

22     return std::noop_coroutine();
23 }

```

When Co_Task ends, we:

1. check if someone waits for us
2. decrement wait counter by 1
3. see if we are the last one and resume awaiting coroutine

With this change, awaiting a single Co_Task requires setting the counter:

```

1 struct Co_Await
2 {
3     Co_Task<T> _task;
4     std::int32_t _wait_count = 1;
5     std::coroutine_handle<> Co_Await::await_suspend(
6         std::coroutine_handle<> waiting_coro) noexcept
7     {
8         promise_type& promise = _task._coro.promise();
9         promise._wait_count = &_wait_count; // 1
10        promise._waiting_coro = waiting_coro;
11        return _task._coro; // resume us
12    }
13 };

```

For awaiting of N coroutines/tasks, we start with CO_await_all():

```

1 template<typename... Ts>
2 static auto CO_await_all(Co_Task<Ts>&&... tasks)
3 {
4     static_assert(sizeof...(Ts) > 0);
5     using Is = std::index_sequence_for<Ts...>;
6     return Co_Await_All<Is, Ts...>{._tasks{std::move(tasks)...}};
7 }

```

It accepts variadic number of any tasks, consumes all of them and returns an awaiter - Co_Await_All. We store all of the Co_Tasks and when wait completes, return a tuple of all of the results:

```

1 template<auto... Is, typename... Ts>
2 struct Co_Await_All<std::index_sequence<Is...>, Ts...>
3 {
4     std::tuple<Co_Task<Ts>...> _tasks;
5     std::int32_t _wait_count = sizeof...(Ts);
6     bool await_ready()
7     {
8         // assert(_tasks[I].is_in_progress()...);
9         return false;
10    }

```

```

11     auto await_resume()
12     {
13         return std::tuple<Ts...>{std::get<Is>(_tasks).get_once()...};
14     }
15 };

```

When awaiting starts, we (a) setup wait_count=N and (b) resume all of the tasks, also linking awaiting coroutine - as in the regular single-task await:

```

1 void Co_Await_All::await_suspend(std::coroutine_handle<> waiting_coro)
2 {
3     auto handle = [&](auto& Task)
4     {
5         auto& promise = Task._coro.promise();
6         promise._wait_count = &_wait_count;
7         promise._waiting_coro = waiting_coro;
8         Task._coro.resume();
9     };
10
11     (handle(std::get<Is>(_tasks)), ...);
12 }

```

Finally, we can await several concurrently running Co_Tasks:

```

1 static Co_Task<int> coro_get(int v)
2 {
3     co_return v;
4 }
5
6 static Co_Task<int> coro_main()
7 {
8     auto [v1, v2] = co_await CO_await_all(coro_get(2), coro_get(3));
9     co_return (v1 + v2);
10 }
11
12 int main()
13 {
14     Co_Task<int> task = coro_main();
15     task.resume();
16     std::println("{} ", task.get_once());
17 }

```


coroutines on top polling tasks

7 # building Fibers API

CODE: CH0x_fiber_basic

(Win32) Fibers materials:

- Using Fibers
- Fibers: the Most Elegant Windows API

In short, we'd like to be able to write something like this:

```
1 const std::string response = FF_await_get(  
2     curl_async, "localhost:5001/file1.txt");  
3 // use `response` as a usual variable, no callbacks
```

to make an asynchronous request. No callbacks, no special keywords.

While “Using Fibers” example above is nice, it's still overly complicated to get basic idea in a simpler form.

Going with Win32 Fibers, short intro is:

- fibers allow to suspend and resume execution at any given point inside a function
- they are stackful coroutines, as opposed to C++20 coroutines that are stackless
- they implement symmetric coroutines (same as C++20 coroutines); we'll build asymmetric coroutines on top of Fibers
- it should be trivial to ifdef POSIX implementation using deprecated `ucontext.h`

and the general idea is:

- you create a fiber with `::CreateFiber()` API; it's suspended
- you switch to/activate/run a fiber with `::SwitchToFiber()` call
- you can switch only between fibers; so everything must be a fiber

Last point is more specific to Win32 API:

- from within a `main()` entry point we are in a thread context
- once `::CreateFiber()` gets you a fiber to switch to, (main) thread needs to be converted to a fiber with a call to `::ConvertThreadToFiber()`

But, ignoring thread-to-fiber conversion, we just (a) create N fibers and (b) switch an execution between them. Worth mentioning: fibers still execute withing a thread context, meaning - context switches (between threads) still happen while fiber is executing.

Below, we go with a `Fiber` class that encapsulates all the system APIs above. We'd like to have a working code that may look like this:

```

1 void Fiber::run()
2 {
3     std::println("fiber1");
4     suspend();
5     std::println("fiber2");
6 }
7
8 int main()
9 {
10     Fiber fiber;
11     std::println("main1");
12     fiber.resume();
13     std::println("main2");
14     fiber.resume();
15     std::println("main3");
16 }

```

and prints:

```

1 main1
2 fiber1
3 main2
4 fiber2
5 main3

```

7.1 # basic Fiber

CODE: CH0x_fiber_basic

We start with a Fiber class that allocates a fiber:

```

1 struct Fiber
2 {
3     void* _fiber = nullptr;
4     void* _parent_fiber = nullptr;
5
6     Fiber(const Fiber&) = delete;
7     Fiber()
8     {
9         _fiber = ::CreateFiber(
10             0 // default stack size
11             , &FiberProc
12             , this); // parameter to FiberProc
13         assert(_fiber);
14     }
15     ~Fiber()
16     {

```

```

17         ::DeleteFiber(_fiber);
18         _fiber = nullptr;
19     }

```

To make our lives easier, there are few assumptions and simplifications:

- we are on x64 system, so there is no need to use extended Fibers API
- we assume LPVOID is void* so no Win32 API types are used (great for headers)
- and, given x64, WINAPI/_stdcall can be omitted, so callbacks passed to Win32 API can have simple C++ declarations

That gives us next include:

```

1  #include <Windows.h>
2  // Note: the floating-point state on x86 systems is not preserved.
3  // If there is need to support x86, Fiber's Ex-tended API must be used.
4  #if !defined(_WIN64)
5  #   error Fiber implementation does not support x86 systems.
6  #endif
7  #include <type_traits>
8  static_assert(std::is_same_v<LPVOID, void*>);

```

Next, while Win32 Fibers can switch from any one Fiber to any other Fiber, we simplify and implement a Fiber that can be resumed, but when suspends - goes to the point of last resume (its parent). Hence, void* _parent_fiber.

FiberProc() we pass to ::CreateFiber() gets a reference to a given Fiber instance and runs it. FiberProc must never end, hence a while loop and a suspend if a call to run() ends:

```

1  static void FiberProc(void* parameter)
2  {
3      assert(parameter);
4      Fiber& self = *static_cast<Fiber*>(parameter);
5      while (true)
6      {
7          self.run();
8          self.suspend();
9      }
10 }

```

To suspend a Fiber is to switch to our parent fiber who did resume us:

```

1  void suspend()
2  {
3      assert(_parent_fiber);
4      void* switch_to_fiber = _parent_fiber;
5      _parent_fiber = nullptr;

```

```

6     ::SwitchToFiber(switch_to_fiber);
7 }

```

To resume a Fiber, we simply call `::SwitchToFiber()` for a `_fiber` we created. However, since when suspending, we need to know how to switch back, we remember current fiber as our parent:

```

1 void resume()
2 {
3     assert(_parent_fiber == nullptr);
4     _parent_fiber = ::GetCurrentFiber();
5     ::SwitchToFiber(_fiber);
6 }

```

Remember, `.resume()` is, basically, a first call to a Fiber from within a main function/thread. That means that `::GetCurrentFiber()` is invoked in a context of `main()`:

```

1 int main()
2 {
3     Fiber fiber;
4     fiber.resume(); // call to ::GetCurrentFiber()??

```

To make that work, specifically for Win32 API, main thread needs to become a Fiber. This is what we do by having a simple RAII class:

```

1 struct Fiber::Boot
2 {
3     Boot(const Boot&) = delete;
4     Boot()
5     {
6         const void* fiber = ::ConvertThreadToFiber(nullptr);
7         assert(fiber);
8     }
9     ~Boot()
10    {
11        const auto ok = ::ConvertFiberToThread();
12        assert(ok);
13    }
14 };

```

Hence, main and/or any other thread that may use Fibers, needs to scope `Fiber::Boot` variable on top:

```

1 int main()
2 {
3     Fiber::Boot _;
4

```

```

5     Fiber fiber;
6     std::println("main1");
7     fiber.resume();
8     std::println("main2");
9     fiber.resume();
10    std::println("main3");
11 }

```

It's possible to avoid that by calling `::ConvertThreadToFiber()` on each and every call to `Fiber::resume()`; choose what you like more.

Given a `main()` above, we:

- create a fiber, which is suspended initially
- print “main1”
- first call to `.resume()` switches us back to `FiberProc` that invokes `Fiber::run()`:

```

1 void Fiber::run()
2 {
3     std::println("fiber1");
4     suspend();
5     std::println("fiber2");
6 }

```

that:

- prints “fiber1”
- suspends itself, which switches back to `main` (our parent fiber)
- `main` prints “main2”, resumes fiber
- fiber prints “fiber2”, exits `run()`, but immediately suspends itself
- `main` prints “main3”

All at once:

```

main1
fiber1
main2
fiber2
main3

```

7.2 # switching between Fibers

CODE: CH0x_fiber_switch

Section above shows how we can switch from a `main` to a different `Fiber`. `Fiber` on it's own, when suspended, switches back to its invoker/resumer. However, instead of `suspend`, `Fiber` can switch execution to a different `Fiber`. While not

quite used anywhere else, lets show the possibility. Our `Fiber::run()` must be able to run different code; lets inject any user-defined callback and run it:

```
1 struct Fiber
2 {
3     // For an example:
4     std::function<void ()> _callback;
5     // ...
6 };
7
8 void Fiber::run()
9 {
10     assert(_callback);
11     _callback();
12 }
```

Where main can now drive 2 Fibers at once:

```
1 void Fiber::run()
2 {
3     assert(_callback);
4     _callback();
5 }
6
7 int main()
8 {
9     Fiber::Boot _;
10
11     Fiber fiber1;
12     Fiber fiber2;
13     fiber1._callback = [&fiber2, self = &fiber1]()
14     {
15         std::println("fiber1: start");
16         fiber2.resume(); // switch to fiber2
17         std::println("fiber1: END");
18         self->suspend(); // switch to main (=our parent)
19     };
20     fiber2._callback = [self = &fiber2]()
21     {
22         std::println("fiber2: start");
23         self->suspend(); // switch back to fiber1 (=our parent)
24         std::println("fiber2: END");
25         self->suspend(); // switch to main (=our parent)
26     };
27     std::println("main1");
28     fiber1.resume();
29     std::println("main2");
```

```

30     fiber2.resume();
31     std::println("main3");
32 }

```

we:

- create 2 fibers; they are suspended
- print “main1”
- resume fiber1, that starts run(), that prints “fiber1: start”
- instead of suspend/switch to main, we then resume/switch to fiber2
- fiber2 resume prints “fiber2: start” initially; then
- we suspend fiber2
- that brings us back to our parent = fiber1
- print “fiber1: END”
- fiber1 suspends itself, that switches back to main
- print “main2”
- main resumes fiber2 which executes last print
- fiber2 prints “fiber2: END”, ends execution by suspending itself
- fiber2 suspend brings back to main
- main prints “main3”

which prints:

```

main1
fiber1: start
fiber2: start
fiber1: END
main2
fiber2: END
main3

```

so, while we used Win32 Fibers to implement asymmetric coroutines:

- we can still freely switch between Fiber(s)
- it does not matter if Fiber was resumed/activated from a thread (main) or the other Fiber
- the act of switching to/resuming is to give a possibility to execute
- this is similar to C++20 coroutines with its `.resume()`
- nothing magically “runs” in the background; there should be a scheduler that resumes or switches between fibers; same is true for C++20 coroutines
- we were able to interleave execution of 3 fibers: main, fiber1, fiber2 - all within one system thread; there are no multiple other threads
- fibers are executed concurrently within main thread

Note, how while executing `fiber2` we (a) were resumed from 2 different contexts and (b) changed the parent in meantime, switching to different contexts:

```

1 fiber2._callback = [self = &fiber2]()
2 {

```

```

3     // ... from fiber1
4     self->suspend(); // switch back to fiber1 (=our parent)
5     // ... from main
6     self->suspend(); // switch to main (=our parent)
7 };

```

7.3 # tasks for Fibers

C++20 coroutines allow for a function to return a value:

```

1 co::Tast<int> MyCoroutine()
2 {
3     co_await Request();
4     co_return 1;
5 }
6
7 int main()
8 {
9     co::Tast<int> task = MyCoroutine();
10    // use a task
11 }

```

We need to build something similar on top of `Fiber` class:

```

1 int MyFiber()
2 {
3     this_fiber::suspend();
4     return 1;
5 }
6
7 int main()
8 {
9     FiberTask<int> task = FF_async([] { return MyFiber(); });
10    // use a task
11 }

```

Note how:

- any mention of a `Fiber` goes away
- there is nice interface to launch a new `Fiber`
- we can suspend itself within an execution context; and
- `MyFiber()` simply returns naked `int`; there is no need to mark every coroutine with task-like return type.

In addition, under the hood:

- we use `Fibers` pool to preallocate `N` fibers and reuse them
- there is simple `Fibers` scheduler to drive our `Fibers` execution

- use exceptions to allow cancellable fiber tasks

Last one is presented just to showcase one of the possibilities; not required for `CURL_async_get()` Fiber wrapper.

7.4 # cancellable Fiber task with a generic callback

CODE: `CH0x_fiber_callback`

First, we start with replacing hardcoded `Fiber::run()` with a generic version that uses next interface:

```

1 // To separate `FiberTask` implementation from an actual `Fiber`.
2 struct FiberCallbackBase
3 {
4     void run()      { return do_run();    }
5     void resume() { return do_resume(); }
6
7     FiberCallbackBase() noexcept = default;
8     FiberCallbackBase(const FiberCallbackBase& rhs) = delete;
9 protected:
10    ~FiberCallbackBase() noexcept = default;
11    virtual void do_run() = 0;
12    virtual void do_resume() {} // optional
13 };

```

Where we allow a user to `run()` anything and also allow to be notified that Fiber is resumed. This is needed to implement a task cancelation: when canceled, user's defined `resume()` throws an exception to unwind everything in a context of fiber execution. Hence, our `run()` implementation becomes:

```

1 static void FiberProc(void* parameter)
2 {
3     Fiber& self = *static_cast<Fiber*>(parameter);
4     while (true)
5     {
6         self.run();
7         self.suspend();
8     }
9 }
10
11 void Fiber::run()
12 {
13     try
14     {
15         _callback->resume();
16         _callback->run();
17     }

```

```

18     catch (...)
19     {
20         _exception = std::current_exception();
21     }
22     _callback = nullptr;
23 }

```

where `_callback` and `_exception` are:

```

1 struct Fiber
2 {
3     void* _fiber = nullptr;
4     void* _parent_fiber = nullptr;
5     FiberCallbackBase* _callback = nullptr;
6     std::exception_ptr _exception;
7     // ...
8 };

```

so, when `run()` enters we (a) notify a user it got resumed (first time) and (b) run everything. If exception is thrown, we just remember it and clean-up our current `_callback` - the execution is done and we go to suspend.

What is `_callback`? This is something user can set on a `Fiber` instance allocated from a `FiberPool` (not shown yet). This is going to be done by a `FiberTask` under the hood. For now, we do everything manually:

```

1 void Fiber::set_callback(FiberCallbackBase& callback)
2 {
3     assert(_callback == nullptr);
4     _callback = &callback;
5     _exception = {};
6 }
7
8 struct MyCallback : FiberCallbackBase
9 {
10     virtual void do_run() override
11     {
12         std::println("fiber1");
13     }
14 };
15
16 int main()
17 {
18     Fiber::Boot _;
19
20     Fiber fiber;
21     MyCallback callback;

```

```

22     fiber.set_callback(callback);
23     std::println("main1");
24     fiber.resume();
25     std::println("main2");
26 }

```

For now, we just made everything we had before more complicated. But the difference is that `Fiber::run()` now is generic and can run anything user-defined.

To support exceptions (cancellation) - the rest of `FiberCallbackBase` interface - our old implementation for `suspend()` needs to be tweak:

```

1 void suspend()
2 {
3     assert(_parent_fiber);
4     void* switch_to_fiber = _parent_fiber;
5     _parent_fiber = nullptr;
6     ::SwitchToFiber(switch_to_fiber);
7     assert(_callback);
8     _callback->resume();
9 }

```

So once `::SwitchToFiber()` returns - meaning other Fiber was running and we are resumed, we notify a user on a new `resume()`.

Finally, to check that Fiber is doing something (either running or suspended from within a user code), we expose next function:

```

1 bool is_busy() const
2 {
3     if (_exception)
4     {
5         std::rethrow_exception(_exception);
6     }
7     return !_callback;
8 }

```

It does cover 2 things: (1) allows to see Fiber has valid user callback and (2) allows to throw any Fiber exception to a user. A bit weird, but does the job.

Overall, the `Fiber` use for a single `Task` - becomes:

1. allocate a new Fiber from a pool
2. set a new callback
3. resume/run the fiber to an end (`is_busy() == false`)
4. return Fiber to the pool
5. repeat for a new `Task`

Note, that from within a `Task` or `FiberCallbackBase`, there is no access to a `Fiber`. How can we suspend a `Task` then? Following `std::this_thread` convention, we

have:

```
1 namespace this_fiber
2 {
3     void suspend()
4     {
5         assert(::IsThreadAFiber());
6         void* fiber_data = ::GetFiberData();
7         assert(fiber_data);
8         Fiber& self = *static_cast<Fiber*>(fiber_data);
9         self.suspend();
10    }
11 } // namespace this_fiber
```

That allows to simply do `this_fiber::suspend()` to give up task execution and be re-scheduled later.

Linking all the pieces together, low-level Fiber Task may look like this:

```
1 struct MyFiberTask : FiberCallbackBase
2 {
3     bool _cancel = false;
4
5     virtual void do_run() override
6     {
7         std::println("fiber1");
8         this_fiber::suspend();
9         std::println("fiber2");
10    }
11
12    virtual void do_resume() override
13    {
14        if (_cancel)
15        {
16            _cancel = false;
17            throw std::exception("canceled");
18        }
19    }
20 };
21
22 int main()
23 {
24     Fiber::Boot _;
25
26     Fiber fiber;
27     MyFiberTask task;
28     fiber.set_callback(task);
```

```

29     std::println("main1");
30     fiber.resume();
31     std::println("main2");
32     task._cancel = true;
33     fiber.resume();
34     std::println("main3");
35 }

```

This is going to be wrapped into nice `FF_async()` interface later. Interesting bit here is that we cancel fiber task in the middle of its execution and “fiber2” console line is not printed:

```

main1
fiber1
main2
main3

```

7.5 # basic FiberPool

CODE: CH0x_fiber_task

The idea is simple: fiber task can be small and short-lived, there is no point allocating (and deallocating) a system Fiber every time task is created. Hence, we preallocate a pool of N (=128) system Fibers that are always alive and reuse them. That means:

- there can be up to N Fiber(s) alive; and
- all system resources are allocated once and never released

FiberPool interface may look like this:

```

1 struct FiberPool
2 {
3     using Handle = std::size_t;
4     explicit FiberPool(std::size_t size) noexcept
5
6     Handle allocate(FiberCallbackBase& callback);
7     void free(Handle handle);
8
9     void suspend(Handle handle);
10    void resume(Handle handle);
11    bool is_busy(Handle handle) const;
12
13 private:
14     std::unique_ptr<Fiber[]> _fibers;
15     std::size_t _size = 0;
16 };

```

where `Fiber` instance is hidden from a user and exposed only with a `Handle`. So we can manage an array of fixed size internally:

```
1 explicit FiberPool::FiberPool(std::size_t size) noexcept
2 {
3     assert(size > 0);
4     _fibers.reset(new(std::nothrow) Fiber[size]);
5     assert(_fibers.get());
6     _size = size;
7 }
```

To create a `Fiber` is to invoke an `allocate()`. Note, we go as dumb and as simple as possible - doing linear search to find first free `Fiber`. It all could be made better, having basic free list allocator for indices as one way to go about it:

```
1 Handle FiberPool::allocate(FiberCallbackBase& callback)
2 {
3     auto find_first_free_handle = [this]()
4     {
5         for (std::size_t i = 0; i < _size; ++i)
6         {
7             if (_fibers[i]._callback == nullptr)
8             {
9                 return Handle(i);
10            }
11        }
12        assert(false);
13        return Handle(-1);
14    };
15    const Handle handle = find_first_free_handle();
16    Fiber& fiber = _fibers[handle];
17    fiber.set_callback(callback);
18    return handle;
19 }
```

`FiberPool::free()` is, basically, no-op. We assert `Fiber` returned is in “complete” state - finished execution:

```
1 void FiberPool::free(Handle handle)
2 {
3     assert(handle < _size);
4     assert(_fibers[handle]._callback == nullptr);
5 }
```

Next, `suspend()`, `resume()` and `is_busy()` are just a wrappers around `Fiber` API, but for a `Handle`:

```

1 void FiberPool::suspend(Handle handle)
2 {
3     assert(handle < _size);
4     _fibers[handle].suspend();
5 }

```

With all this, our previous example running fiber task becomes:

```

1 int main()
2 {
3     Fiber::Boot _;
4     FiberPool fiber_pool{128};
5
6     MyFiberTask task;
7     FiberPool::Handle fiber = fiber_pool.allocate(task);
8     fiber_pool.resume(fiber);
9     fiber_pool.free(fiber);
10 }

```

Now, lets get rid of manually created MyFiberTask and make FiberTask<T> possible.

7.6 # FiberTask

CODE: CH0x_fiber_task

We'd like to be able to write something like this:

```

1 int MyFiber()
2 {
3     this_fiber::suspend();
4     return 1;
5 }
6
7 int main()
8 {
9     FiberTaskScheduler scheduler;
10    FiberTask<int> task = FF_async(scheduler, [] { return MyFiber(); });
11    // ...
12    // scheduler.schedule();
13 }

```

There are several moving parts:

1. we need type-erased implementation of FiberTask that can consume any user-defined lambda/callback, work with any return type T; and
2. we need a scheduler that knows how to drive tasks/fibers execution

FiberTask<T> allocates a Fiber, runs a given callable/lambda and stores the

result. Given `FiberTask` can be destroyed by a user any time, the way we handle this is to ensure that actual `FiberTask` state is owned by `FiberTaskScheduler`: if `FiberTask` is destroyed while still running, we cancel `Fiber` and free it later, once the execution is done.

Interestingly, there could be up to N active Fibers (`FiberPool` capacity), but `FiberTasks` count owned by a user can be larger. Consider next code:

```

1  FiberTask<void> m_tasks[N];
2  for (...) { m_tasks[i] = FF_async(...) }
3  // N Fiber tasks completes
4  //
5  // User still owns N m_tasks and may query
6  // the status and result - any time later.
```

Going with more optimized implementation, we could:

- preallocate N system Fibers
- preallocate a memory for up to M ($\geq N$) fiber Tasks
- each fiber Task could be limited in size (let 256 bytes); and
- each Task could free Fiber as soon as task ends

i.e., Fiber and Task lifetime could be different and everything is preallocated; meaning no allocations at runtime.

For simpler implementation, we (a) bind Fiber lifetime to a Task lifetime: Fiber is released only when Task is destroyed; and (b) Task allocates on creation, going with more standard code for type-erased implementation.

With this in mind, we start with `FiberTask_Any`:

```

1  struct FiberTask_Any : public FiberCallbackBase
2  {
3      explicit FiberTask_Any(FiberPool& fiber_pool);
4      virtual ~FiberTask_Any() noexcept;
5
6      bool is_completed() const;
7      void execute();
8      void cancel();
9
10     private:
11         virtual void do_resume() override;
12
13     protected:
14         FiberPool& _fiber_pool;
15         FiberPool::Handle _fiber;
16         bool _cancelled = false;
17 };
```

that implements `FiberCallbackBase` interface and can be cancelled, but knows

nothing on how to store the result of execution:

```
1  FiberTask_Any::FiberTask_Any(FiberPool& fiber_pool)
2      : _fiber_pool(fiber_pool)
3      , _fiber(fiber_pool.allocate(*this))
4  {
5  }
6  FiberTask_Any::~FiberTask_Any() noexcept
7  {
8      assert(is_completed());
9      _fiber_pool.free(_fiber);
10 }
```

For cancellation, we throw an exception when Fiber is resumed:

```
1  class Exception_FiberTaskCancelled : public std::exception
2  {
3  };
4
5  void FiberTask_Any::cancel()
6  {
7      assert(_cancelled == false);
8      _cancelled = true;
9  }
10
11 void FiberTask_Any::do_resume()
12 {
13     if (_cancelled)
14     {
15         _cancelled = false;
16         throw Exception_FiberTaskCancelled{};
17     }
18 }
```

To execute task is to resume a Fiber:

```
1  void FiberTask_Any::execute()
2  {
3      assert(is_completed() == false);
4      _fiber_pool.resume(_fiber);
5  }
```

Finally, `is_completed()` check looks for a Fiber that ended the execution and/or has an exception thrown:

```
1  bool FiberTask_Any::is_completed() const
2  {
3      try
4      {
```

```

5         return (_fiber_pool.is_busy(_fiber) == false);
6     }
7     catch (...)
8     {
9     }
10    return true;
11 }

```

To handle void and non-void return types, we introduce `FiberTask_WithResult`:

```

1  template<typename R>
2  struct FiberTask_WithResult : public FiberTask_Any
3  {
4  public:
5      using FiberTask_Any::FiberTask_Any;
6      const R& get() const;
7      R&& get_once()
8      {
9          const R& r = static_cast<const FiberTask_WithResult&>(*this).get();
10         return std::move(const_cast<R&>(r));
11     }
12 protected:
13     // In case return type is not DefaultConstructible.
14     std::variant<std::monostate, R> _storage;
15 };
16
17 template<>
18 struct FiberTask_WithResult<void> : public FiberTask_Any
19 {
20 public:
21     using FiberTask_Any::FiberTask_Any;
22     void get() const;
23     void get_once()
24     {
25         return static_cast<const FiberTask_WithResult&>(*this).get();
26     }
27 };

```

where non-void `FiberTask` places the result in the `_storage` data member. `get_once()` is just a `get()` that give rvalue reference back so it could be moved from. `get()` implementation is just:

```

1  const R& get() const
2  {
3      // Populate exception, if any.
4      const bool running = _fiber_pool.is_busy(_fiber);
5      assert(running == false);

```

```

6     assert(_storage.index() == 1);
7     return std::get<1>(_storage);
8 }

```

Note how we must check Fiber's `is_busy()` to populate any exception to the user if something was throws during Fiber execution.

Finally, to run a lambda, we go with `FiberTask_Callable`:

```

1  template<typename R, typename C>
2  struct FiberTask_Callable : public FiberTask_WithResult<R>
3  {
4      using Base = FiberTask_WithResult<R>;
5  public:
6      explicit FiberTask_Callable(C&& callable, FiberPool& fiber_pool)
7          : Base(fiber_pool)
8            , _callable(std::move(callable))
9      {
10     }
11 private:
12     virtual void do_run() override
13     {
14         if constexpr (std::is_same_v<void, R>)
15         {
16             _callable();
17         }
18         else
19         {
20             this->_storage.template emplace<1>(_callable());
21         }
22     }
23 private:
24     C _callable;
25 };

```

`FiberTask_Callable` is what's going to be allocated on every `FF_async()` call that creates a `FiberTask<T>`:

```

1  template<typename R>
2  struct FiberTask
3  {
4      using Task = FiberTask_WithResult<R>;
5
6      template<typename C>
7      explicit FiberTask(C&& callable, FiberTaskScheduler& scheduler) noexcept
8          : _scheduler(&scheduler)
9      {

```

```

10         using TaskCallable = FiberTask_Callable<R, std::remove_cvref_t<C>>;
11         TaskCallable* task = new(std::nothrow) TaskCallable(
12             std::forward<C>(callable), scheduler._fiber_pool);
13         assert(task);
14         scheduler.add_fiber_task(std::unique_ptr<FiberTask_Any>(task));
15         _task = task;
16     }
17     ~FiberTask() noexcept
18     {
19         destroy_once();
20     }
21     void destroy_once() noexcept
22     {
23         if (_scheduler)
24         {
25             assert(_task);
26             _scheduler->remove_fiber_task(*_task);
27             _scheduler = nullptr;
28             _task = nullptr;
29         }
30     }
31     decltype(auto) get() const;
32     decltype(auto) get_once();
33     // ...
34     FiberTaskScheduler* _scheduler = nullptr;
35     Task* _task = nullptr;
36 };
37
38 template<typename C>
39 auto FF_async(FiberTaskScheduler& scheduler, C&& callable)
40 {
41     using Task = FiberTask<std::invoke_result_t<C>>;
42     return Task{std::forward<C>(callable), scheduler};
43 }

```

_task itself is owned by a FiberTaskScheduler:

```

1 struct FiberTaskScheduler
2 {
3     FiberPool& _fiber_pool;
4     std::vector<std::unique_ptr<FiberTask_Any>> _tasks;
5     std::vector<FiberTask_Any*> _tasks_to_remove;
6
7     void add_fiber_task(std::unique_ptr<FiberTask_Any>&& task)
8     {
9         assert(task.get());

```

```

10     _tasks.push_back(std::move(task));
11 }
12 void remove_fiber_task(FiberTask_Any& task)
13 {
14     if (task.is_completed() == false)
15     {
16         task.cancel();
17     }
18     _tasks_to_remove.push_back(&task);
19 }

```

Note how removing a task is also a cancel if needed.

We are left with FiberTaskScheduler execution, which is its `schedule()`:

```

1 void FiberTaskScheduler::schedule()
2 {
3     while (schedule_once()) {}
4 }
5
6 bool FiberTaskScheduler::schedule_once()
7 {
8     bool repeat = false;
9     auto tasks = std::move(_tasks);
10    for (auto it = tasks.rbegin(); it != tasks.rend(); ++it)
11    {
12        std::unique_ptr<FiberTask_Any>& task = *it;
13        assert(task);
14        if (task->is_completed() == false)
15        {
16            task->execute();
17            repeat |= task->is_completed();
18            continue;
19        }
20        auto it_remove = std::find(_tasks_to_remove.begin()
21            , _tasks_to_remove.end(), task.get());
22        if (it_remove == _tasks_to_remove.end())
23        {
24            continue;
25        }
26        task.reset();
27    }
28    for (std::unique_ptr<FiberTask_Any>& task : tasks)
29    {
30        if (task)
31        {

```

```

32         _tasks.push_back(std::move(task));
33     }
34 }
35 return repeat;
36 }

```

There are 2 moments worth mentioning:

1. tasks are executed from the end - this is needed to execute child tasks first so parent tasks that wait can have progress
2. if any task is completed, we schedule a loop again - again so any parent tasks can complete if child tasks complete

Other than that, we just `.execute()` every task and remove it when completed and was removed.

Finally, running a `FiberTask` is:

```

1  int MyFiber()
2  {
3      this_fiber::suspend();
4      return 1;
5  }
6
7  int main()
8  {
9      Fiber::Boot _;
10     FiberPool fiber_pool{128};
11     FiberTaskScheduler fibers_scheduler{fiber_pool};
12
13     FiberTask<int> task = FF_async(fibers_scheduler, &MyFiber);
14     while (task.is_completed() == false)
15     {
16         fibers_scheduler.schedule();
17     }
18     std::println("{} ", task.get()); // prints "1"
19 }

```

fibers (WIN32) (App_Fibers)

8 # senders

9 # reactive streams